



CEBU INSTITUTE OF TECHNOLOGY
U N I V E R S I T Y

IT342-G1

System Integration and Architecture

System Design Document (SDD)

Project Title: Lost and Found

Prepared By: Tiongzon, Jay Yan

Version: 0.3

Date: 2/28/2026

Status: Final

REVISION HISTORY TABLE

Version	Date	Author	Changes Made	Status
0.1	02/21/26	Tiongzon, Jay Yan	Initial draft	Draft
0.2	02/28/26	Tiongzon, Jay Yan	Added contents for System Architecture, API Contract and Communication Flow, Database Design, UI/UX Design, and Project Plan.	Final
0.3	[Date]	Tiongzon, Jay Yan	Updated database design	Review
0.4	[Date]	Tiongzon, Jay Yan	Added UI/UX designs	Review
0.5	[Date]	Tiongzon, Jay Yan	Incorporated feedback	Revised
0.6	[Date]	Tiongzon, Jay Yan	Final review and corrections	Final
1	[Date]	Tiongzon, Jay Yan	Baseline version for development	Approved

TABLE OF CONTENTS

Contents

EXECUTIVE SUMMARY	4
1.0 INTRODUCTION	5
2.0 FUNCTIONAL REQUIREMENTS SPECIFICATION	5
3.0 NON-FUNCTIONAL REQUIREMENTS	8
4.0 SYSTEM ARCHITECTURE	9
5.0 API CONTRACT & COMMUNICATION	10
Authentication Endpoints	11
User Registration	11
User Login	11
6.0 DATABASE DESIGN	13
7.0 UI/UX DESIGN	14
8.0 PLAN	17

EXECUTIVE SUMMARY

1.1 Project Overview

LostAndFound is a platform designed to streamline the reporting, tracking, and claiming of lost items within a campus or local community. The system utilizes a clear layered architecture integrating a Spring Boot backend API, a Next.js web application, and an Android mobile app to provide a seamless and accessible experience for users trying to recover their belongings.

1.2 Objectives

1. Develop a fully functional MVP with secure authentication, an item catalog (lost and found boards), claim management, and user profiles.
2. Implement a layered architecture using Spring Boot, Next.js, and Android.
3. Create RESTful APIs for communication between all system components
4. Create structured, RESTful API endpoints for communication between all system components.
5. Design a responsive user interface that works consistently across web and mobile platforms.

1.3 Scope

Included Features:

- **Authentication & Security:** User registration and login utilizing JWT Authentication. Passwords will be secured using BCrypt hashing, and the system will include protected routes and a /me endpoint.
- **Social Login:** Google OAuth integration to allow users to sign in quickly, generating a custom JWT upon successful linking to the database.
- **Role-Based Access Control:** API and UI-level role restrictions supporting at least two roles. For example, an Admin can manage all item reports across the system, while a Regular user has limited access to manage only their own posts and claims.
- **Core Business Module:** Full CRUD operations and proper validation for major entities like Items (Lost/Found records) and Claims.
- **System Integration - External API:** Integration with a real public API, such as a Maps API (e.g., Google Maps or Mapbox), to meaningfully display the location where an item was lost or found on a map interface.

- System Integration - File Upload: Server-side file upload functionality allowing users to upload images of found items or proof of ownership (like an ID), linked directly to the database record.
- System Integration - Payment Gateway: Integration with a real payment provider in Test Mode (Sandbox). This fulfills the requirement by implementing an optional "Finder's Reward" system or a "Courier Return Fee" to ship items back to the owner, properly handling both success and failure states.
- System Integration - Email Sending: SMTP integration to send at least one account-related email (e.g., account verification upon registration) and one system notification (e.g., an alert when someone requests to claim your found item).
- Database Architecture: A MySQL relational database utilizing 5 normalized tables (Users, Roles, Items, Claims, and Messages) with proper JPA/Hibernate relationships. API responses strictly use Data Transfer Objects (DTOs) to prevent exposing passwords or sensitive data.
- Real-Time Communication (WebSockets): Integration of Spring Boot WebSockets with STOMP and SockJS to provide instant, live updates for users. This powers both a Global Community Chat and Private 1-on-1 Messaging between finders and claimers without requiring page refreshes.
- Content Moderation & Warning System: A comprehensive Admin Dashboard that allows administrators to issue warning marks to users breaking community guidelines. The system automatically enforces bans on users who accumulate too many warnings, revoking their login access.

Excluded Features:

- Live/Production payment processing (strictly simulated/sandbox).
- Automated physical inventory tracking.
- Push notifications (relying on email instead).
- Advanced analytics dashboard.

1.0 INTRODUCTION

1.1 Purpose

This document serves as the comprehensive design specification for the LostAndFound system. It provides detailed requirements, architectural decisions, API

contracts, database design, and an implementation roadmap to guide development and ensure all components—including the Spring Boot backend, Next.js web frontend, and Android mobile application—integrate seamlessly to meet the final defense requirements.

2.0 FUNCTIONAL REQUIREMENTS SPECIFICATION

2.1 Project Overview

Project Name: LostAndFound

Domain: Campus/Community Utility

Primary Users:

1. Students/Faculty
2. Administrators

Problem Statement: Students and community members need a centralized, fast way to report, track, and claim lost items across campus without relying on fragmented social media posts or physical bulletin boards.

Solution: A streamlined, cross-platform application focusing on logging lost/found items, verifying ownership, and facilitating returns with a consistent experience across web and mobile.

2.2 Core User Journeys

Journey 1: Reporting a Found Item

1. User logs in using Google OAuth or standard credentials.
2. User navigates to "Report Found Item" and enters details.
3. User uploads an image of the found item.
4. User pins the exact location where it was found using the integrated Maps API.
5. Item is posted to the database, and the user receives a confirmation email.

Journey 2: Claiming a Lost Item

1. User searches the platform for their lost item.
2. User finds a match and clicks "File Claim", providing proof of ownership (e.g., ID upload or detailed description).
3. User optionally pays a "Courier/Return Fee" via the Sandbox Payment Gateway to have the item shipped to them.
4. Original finder receives a system notification email about the claim.

Journey 3: Administrator Moderation

1. Admin logs in with admin credentials.
2. Admin views the master list of all reported items and active claims.
3. Admin resolves disputes or deletes inappropriate listings.

2.3 Feature List (MoSCoW)

MUST HAVE

1. User authentication featuring registration, login, logout, and a `/me` endpoint to get the current user.
2. JWT Authentication and password hashing using BCrypt.
3. Role-Based Access Control supporting at least 2 roles (Admin, Regular User) with API and UI-level restrictions.
4. Core Business Module featuring full CRUD operations and proper validation for major entities (e.g., Items, Claims).
5. External API Integration that consumes a real public API (e.g., Google Maps/Mapbox for item locations) and displays the data in the system.
6. Google OAuth Login that saves the user in the database and generates a custom JWT.
7. File Upload allowing users to upload images stored on the server and linked to database records.
8. Payment Gateway Integration using a real payment provider in Test/Sandbox mode to handle success and failure states (e.g., for optional reward fees or shipping).
9. Email Sending (SMTP) to send one account-related email and one system notification email.

SHOULD HAVE

1. Item categories (electronics, clothing, documents) and filtering.
2. Responsive design for all screen sizes.

COULD HAVE

1. Real-Time Feature using WebSockets: Live messaging updates between users via a Global Chat room and Private Direct Messaging channels for coordinating claims, demonstrating low-latency communication.

WON'T HAVE

1. Simulated payment logic (must use real provider test mode).
2. Live/production payment processing.
3. Console-print email mocking.

2.4 Detailed Feature Specifications

Feature: User Authentication & Security

- **Screens:** Registration, Login, Social Auth Redirect.
- **Fields:** Email, Password, Name.
- **Validation:** Email format, uniqueness.
- **API Endpoints:** `POST /auth/register`, `POST /auth/login`, `POST`

`/auth/logout, GET /users/me.`

- **Security:** Protected routes. JWT tokens must be generated after both standard and OAuth logins. Passwords must be hashed with BCrypt.

Feature: Item Board (Core Business Module)

- **Screens:** Item Feed, Item Detail, Create Report.
- **Functions:** Create, Read, Update, Delete lost/found items. Upload images to link to the record.
- **Integration:** Render a map component using a public API to show the item's location.
- **API Endpoints:** `GET /items`, `POST /items`, `PUT /items/{id}`, `DELETE /items/{id}`.

Feature: Claims & Notifications

- **Screens:** Claim Form, Claim Status.
- **Process:** User submits a claim, triggering a system notification email to the item's reporter via SMTP.
- **Payments:** Optional flow to process a finder's reward or shipping fee via Sandbox payment gateway, recording the result in the database.

Feature: Admin Panel

- **Screens:** Dashboard, User Management, Content Moderation.
- **Functions:** Manage all data across the platform.
- **Access Control:** UI and API endpoints restricted to Admin role only.

2.5 Acceptance Criteria

AC-1: Secure Authentication & OAuth

- Given I am a user on the login page,
- When I click "Continue with Google",
- Then my account should be linked in the database,
- And the system must generate a custom JWT for my session,
- And I should receive a welcome email via SMTP.

AC-2: Item Reporting Flow

- Given I am a logged-in Regular user,
- When I create a new "Found Item" report,
- And I upload an image of the item,
- And I select the location using the external Maps API,

- Then the record should be saved with the linked file,
- And displayed on the system feed.

AC-3: Payment Integration

- Given I am claiming an item,
- When I initiate a courier fee payment,
- And I use the real payment provider's test credentials,
- Then the system should handle the success/failure response,
- And record the transaction result in the database.

3.0 NON-FUNCTIONAL REQUIREMENTS

3.1 Performance Requirements

- API response time: ≤ 2 seconds for 95% of requests.
- Web page load time: ≤ 3 seconds on broadband.
- Mobile app cold start: ≤ 3 seconds.
- Database queries complete within 500ms.
- If implementing the optional Real-Time Feature (Bonus), live updates between users should reflect with minimal latency via WebSocket or Polling.

3.2 Security Requirements

- HTTPS for all communications.
- JWT authentication must secure all protected routes/pages.
- Password hashing must be implemented using BCrypt ; no plain-text passwords will be stored in the database.
- Strict Role-Based Access Control (RBAC) must be enforced at both the API-level and UI-level.
- The backend architecture must utilize Data Transfer Objects (DTOs) to ensure passwords and sensitive data are never exposed in API responses.
- A dedicated security configuration layer must be implemented in the backend.
- Global exception handling must be utilized to securely manage errors without exposing backend stack traces to the client.

3.3 Compatibility Requirements

- **Web Client:** Compatible with Chrome, Firefox, Safari, Edge (latest 2 versions).
- **Mobile Client:** Android application supporting API Level 24+ (Android)

- **Screen Sizes:** Responsive design supporting Mobile (360px+), Tablet (768px+), and Desktop (1024px+).

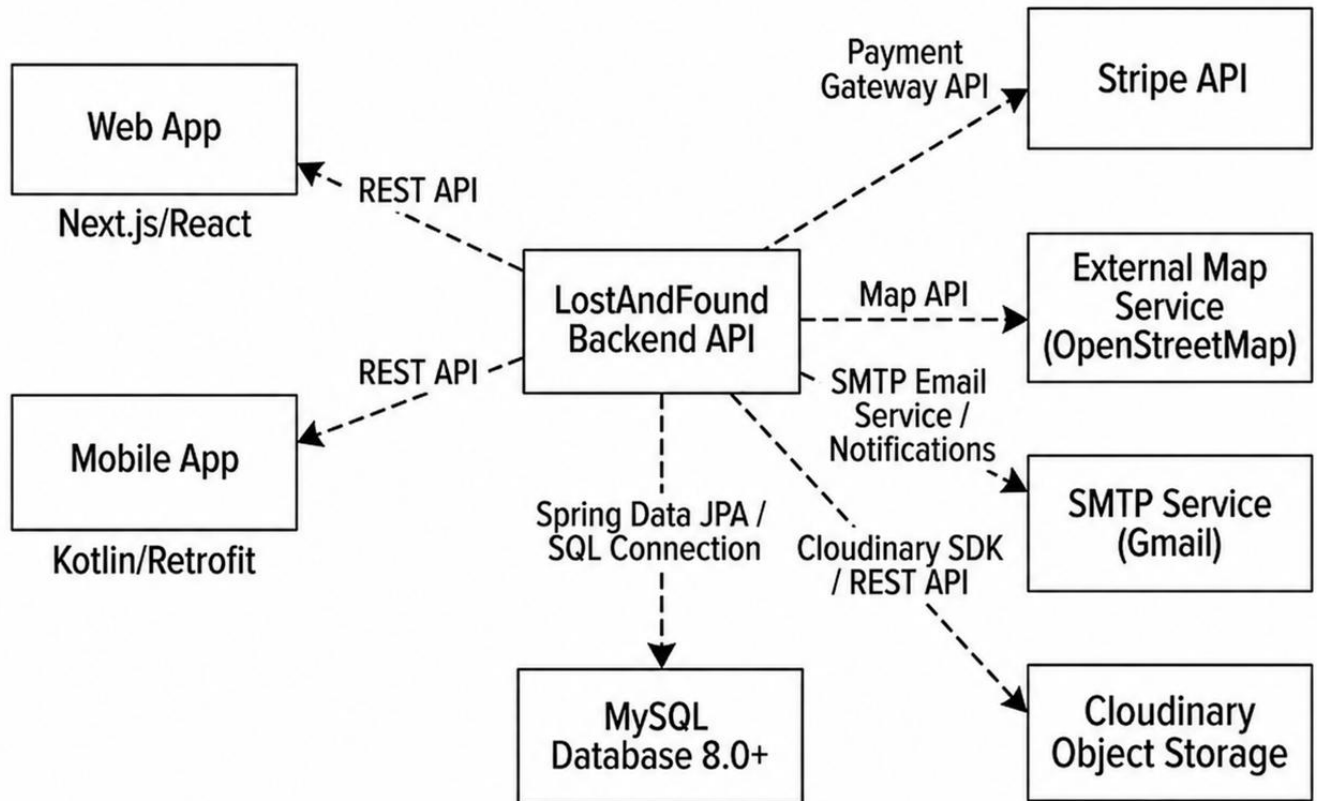
3.4 Usability Requirements

- Users should be able to successfully submit a lost or found item report within 3 minutes.
- Standardized use of proper HTTP status codes and RESTful endpoint naming must be used to ensure clear, predictable communication between the web/mobile clients and the backend.
- UI-level role restrictions must dynamically adapt the interface (e.g., hiding admin panels from regular users to prevent confusion).
- Clear error messages and recovery options must be displayed, driven by the backend's global exception handler.

4.0 SYSTEM ARCHITECTURE

4.1 Component Diagram

Note: This should be a component diagram



Technology Stack:

- **Backend:** Java 17, Spring Boot 3.x, Spring Security (JWT Auth), Spring Data JPA, WebSocket
- **Database:** MySQL 8.0+
- **Web Frontend:** React 18, Next.js, TypeScript, Tailwind CSS, Lucide React
- **Mobile:** Android Kotlin, XML-Based UI Layouts, Retrofit, Room
- **Build Tools:** Maven (Backend), npm (Web), Gradle (Android)

- **Deployment:** Railway / Render (Backend), Vercel (Web), APK (Mobile)

5.0 API CONTRACT & COMMUNICATION

5.1 API Standards

Base URL	https://[server_hostname]:[port]/api/v1
Format	JSON for all requests/responses
Authentication	Bearer token (JWT) in Authorization header
Response Structure	<pre>{ "success": boolean, "data": object null, "error": { "code": string, "message": string, "details": object null }, "timestamp": string }</pre>

5.2 Endpoint Specifications

Authentication Endpoints

User Registration

Description	User Registration
API URL	/auth/register
HTTP Request Method	POST
Format	JSON for all requests/responses
Authentication	None
Request Payload	<pre>{ "email": <email>, "username": <username>, "password": <password> }</pre>

Response Structure	<pre> { "success": boolean, "data": { "message": "Registration success!" }, "error": { "code": string, "message": string, "details": object null }, "timestamp": string } </pre>
--------------------	--

User Login

Description	User Login
API URL	/auth/login
HTTP Method	POST
Format	JSON for all requests/responses
Authentication	None
Request Payload	<pre> { "username": <username>, "password": <password> } </pre>

Response Structure	<pre> { "success": boolean, "data": { "token": <token> }, "error": { "code": string, "message": string, "details": object null }, "timestamp": string } </pre>
--------------------	--

Get Current User

Description	Get current authenticated user profile
API URL	/auth/me
HTTP Method	GET
Format	JSON for all requests/responses
Authentication	Bearer token (JWT)
Request Payload	None

Response Structure	<pre> { "success": boolean, "data": { "id": number, "username": string, "email": string, "role": string, "warningMarks": number, "isBanned": boolean, "avatarUrl": string null }, "error": null, "timestamp": string } </pre>
---------------------------	---

Item (Catalog) Endpoints

Get All Items

Description	Fetch paginated list of items with optional filters
API URL	/items
HTTP Request Method	GET
Format	JSON for all requests/responses
Authentication	None
Request Payload	Query params: page, size, type, category, keyword

Response Structure	<pre> { "success": boolean, "data": { "content": [...ItemObjects], "totalPages": number, "totalElements": number }, "error": null, "timestamp": string } </pre>
--------------------	---

Get Item Details

Description	Fetch specific item details
API URL	/items/{id}
HTTP Request Method	GET
Format	JSON for all requests/responses
Authentication	None
Request Payload	None

Response Structure	<pre> { "success": boolean, "data": { ...ItemObject }, "error": null, "timestamp": string } </pre>
---------------------------	--

Create Item

Description	Report a new lost or found item
API URL	/items
HTTP Request Method	POST
Format	Multipart/form-data (if image included) or JSON
Authentication	Bearer token (JWT)
Request Payload	FormData with fields: title, description, type, category, location, date, image (file)

Response Structure	<pre>{ "success": boolean, "data": { ...ItemObject }, "error": null, "timestamp": string }</pre>
---------------------------	--

Update Item Status

Description	Update item status (e.g., mark as RESOLVED)
API URL	/items/{id}
HTTP Request Method	PATCH
Format	JSON for all requests/responses
Authentication	Bearer token (JWT)
Request Payload	<pre>{ "status": <status>, "type": <type> }</pre>

Response Structure	<pre> { "success": boolean, "data": { ...UpdatedItemObject }, "error": null, "timestamp": string } </pre>
--------------------	---

Claims Endpoints

Submit Claim

Description	Submit a claim for an item with proof
API URL	/claims
HTTP Request Method	POST
Format	Multipart/form-data
Authentication	Bearer token (JWT)
Request Payload	FormData with fields: itemId, proofDescription, proofImage (file)

Response Structure	<pre> { "success": boolean, "data": { ...ClaimObject }, "error": null, "timestamp": string } </pre>
---------------------------	---

Get User Claims

Description	Get all claims made by the authenticated user
API URL	/claims
HTTP Request Method	GET
Format	JSON for all requests/responses
Authentication	Bearer token (JWT)
Request Payload	None

Response Structure	<pre>{ "success": boolean, "data": [...ClaimObjects], "error": null, "timestamp": string }</pre>
--------------------	--

Chat / Messaging Endpoints

Get Global Chat Messages

Description	Retrieve messages for the global community chat
API URL	/chat
HTTP Request Method	GET
Format	JSON for all requests/responses
Authentication	Bearer token (JWT)
Request Payload	None

Response Structure	<pre>{ "success": boolean, "data": [...MessageObjects], "error": null, "timestamp": string }</pre>
--------------------	--

Send Global Message

Description	Send a message to the global chat
API URL	/chat
HTTP Request Method	POST
Format	JSON for all requests/responses
Authentication	Bearer token (JWT)
Request Payload	<pre>{ "content": <messageContent> }</pre>

Response Structure	<pre> { "success": boolean, "data": { ...MessageObject }, "error": null, "timestamp": string } </pre>
--------------------	---

User Profile Endpoints

Update User Avatar

Description	Upload a new profile picture
API URL	/users/me/avatar
HTTP Request Method	PATCH
Format	Multipart/form-data
Authentication	Bearer token (JWT)
Request Payload	FormData with field: file (image)

Response Structure	<pre> { "success": boolean, "data": { "avatarUrl": string }, "error": null, "timestamp": string } </pre>
---------------------------	--

Admin Endpoints

Get All Users

Description	Fetch all users for moderation
API URL	/admin/users
HTTP Request Method	GET
Format	JSON for all requests/responses
Authentication	Bearer token (JWT) - Admin Role Required
Request Payload	None

Response Structure	<pre>{ "success": boolean, "data": [...UserObjects], "error": null, "timestamp": string }</pre>
---------------------------	---

Warn User

Description	Increment a user's warning marks
API URL	/admin/users/{userId}/warn
HTTP Request Method	POST
Format	JSON for all requests/responses
Authentication	Bearer token (JWT) - Admin Role Required
Request Payload	None

Response Structure	<pre> { "success": boolean, "data": { ...UpdatedUserObject }, "error": null, "timestamp": string } </pre>
--------------------	---

Ban/Unban User

Description	Toggle a user's ban status
API URL	/admin/users/{userId}/ban
HTTP Request Method	POST
Format	JSON for all requests/responses
Authentication	Bearer token (JWT) - Admin Role Required
Request Payload	None

Response Structure	<pre> { "success": boolean, "data": { ...UpdatedUserObject }, "error": null, "timestamp": string } </pre>
--------------------	---

Admin Delete Post

Description	Arbitrarily delete any item post for moderation
API URL	/admin/items/{itemId}
HTTP Request Method	DELETE
Format	JSON for all requests/responses
Authentication	Bearer token (JWT) - Admin Role Required
Request Payload	None

Response Structure	<pre>{ "success": boolean, "data": { "message": "Post safely deleted" }, "error": null, "timestamp": string }</pre>
--------------------	---

5.3 Error Handling

HTTP Status Codes

- 200 OK - Successful request
- 201 Created - Resource created
- 400 Bad Request - Invalid input
- 401 Unauthorized - Authentication required/failed
- 403 Forbidden - Insufficient permissions
- 404 Not Found - Resource doesn't exist
- 409 Conflict - Duplicate resource
- 500 Internal Server Error - Server error

Error Code Examples

Example 1	<pre>{ "success": false, "data": null, "error": { "code": "AUTH-001", "message": "Invalid credentials", "details": "Username or password is incorrect" }, "timestamp": "2024-01-28T10:30:00Z" }</pre>
-----------	---

Example 2	<pre> { "success": false, "data": null, "error": { "code": "VALID-001", "message": "Validation failed", "details": { "username": "Username already exists!", "password": "Must be at least 8 characters" } }, "timestamp": "2024-01-28T10:30:00Z" } </pre>
-----------	--

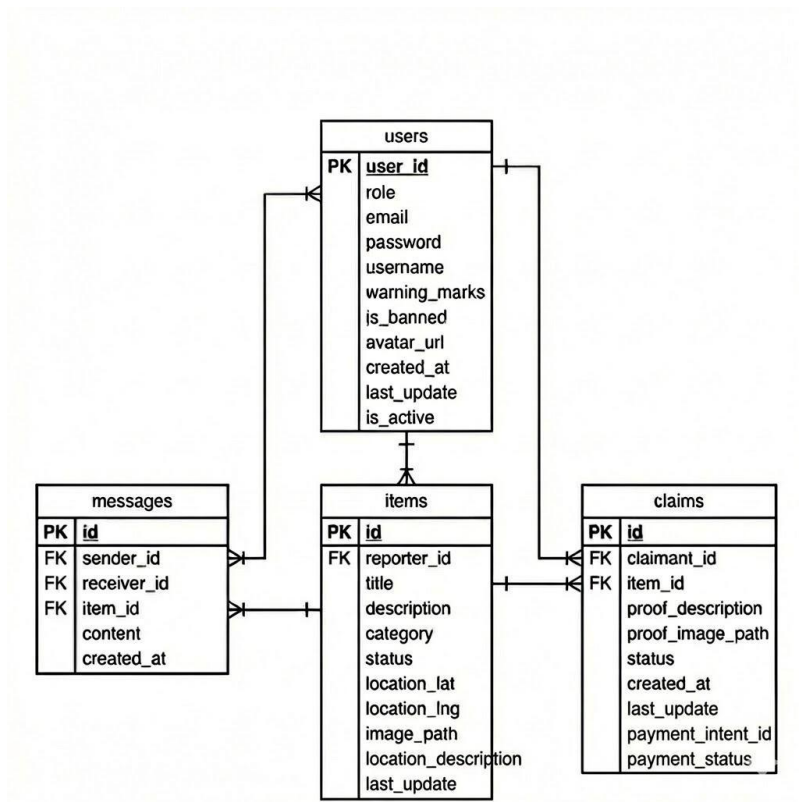
Common Error Codes

- AUTH-001: Invalid credentials
- AUTH-002: Token expired
- AUTH-003: Insufficient permissions
- VALID-001: Validation failed
- DB-001: Resource not found
- DB-002: Duplicate entry
- BUSINESS-001: Insufficient stock
- SYSTEM-001: Internal server error

6.0 DATABASE DESIGN

6.1 Entity Relationship Diagram

Note: This should be an ERD (Database Schema)



Detailed Relationships:

- One-to-Many: User → Items (User can report multiple lost or found items)
- One-to-Many: User → Claims (User can file multiple claims)
- Many-to-One: Claims → Item (Multiple claims can be filed against a single item)
- Many-to-One: Messages → User (Each message has one sender)
- Many-to-Many: User ↔ Roles (A User can have multiple roles, and roles can belong to multiple users)

Key Tables:

1. users - User accounts, authentication details, and moderation status
2. roles - System access roles (e.g., ADMIN, USER)
3. user_roles - Join table connecting Users and Roles
4. items - Catalog of reported Lost and Found items
5. claims - Ownership claims filed against specific items
6. messages - Real-time chat history between users

Table Structure Summary:

- users: id, email, username, password, avatar_url, warning_marks, is_banned, created_at, updated_at
- roles: id, name

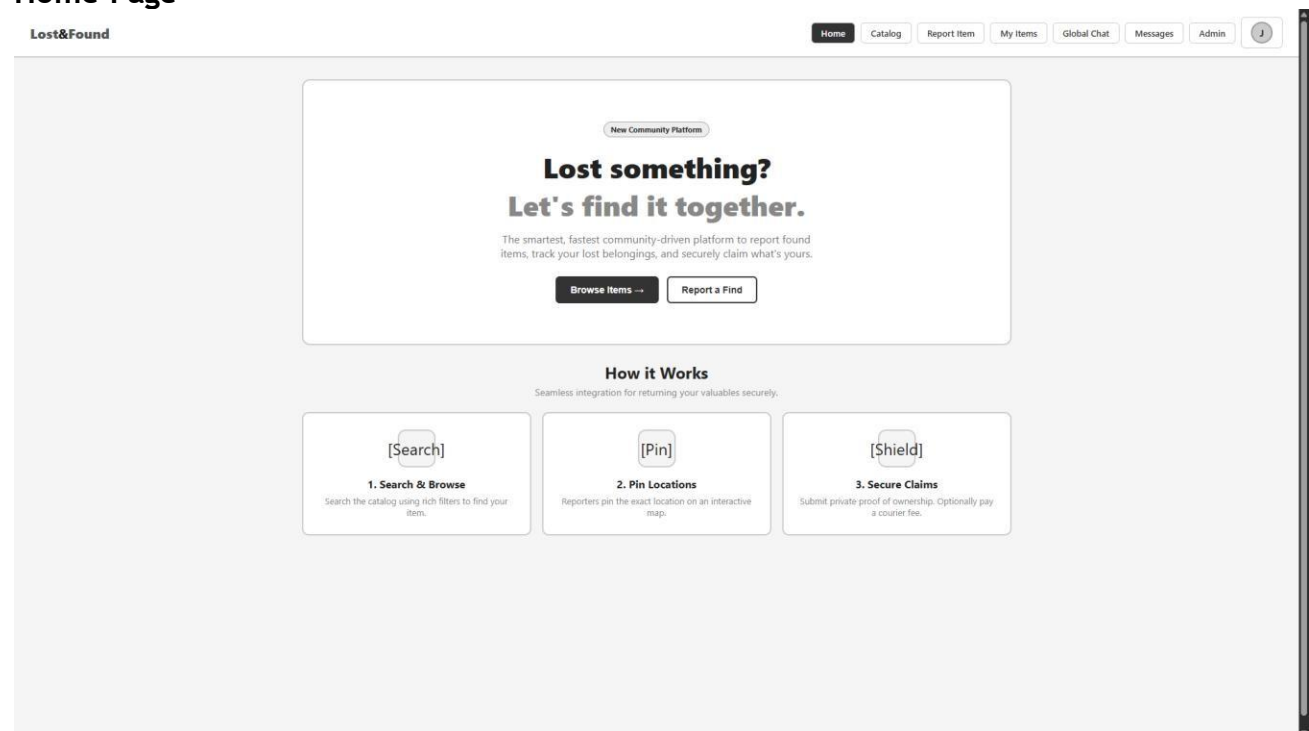
- user_roles: user_id, role_id
- items: id, title, description, status (LOST/FOUND/RESOLVED), category, location_lat, location_lng, location_description, image_path, reporter_id, created_at, last_update
- claims: id, item_id, claimant_id, proof_description, proof_image_path, status (PENDING/APPROVED/REJECTED), payment_status, payment_intent_id, created_at, last_update
- messages: id, sender_id, receiver_id, item_id, content, created_at

7.0 UI/UX DESIGN

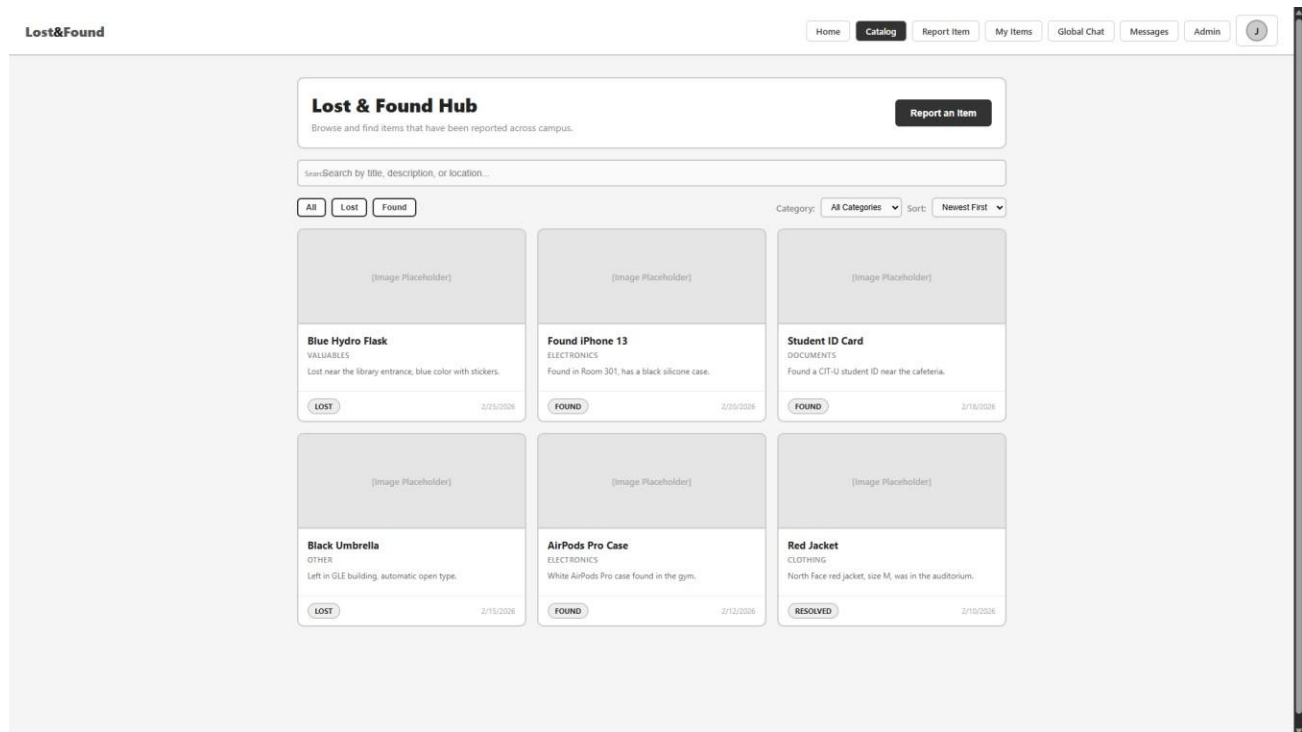
7.1 Web Application Wireframes

Note: This should be wireframes from Figma

Home Page



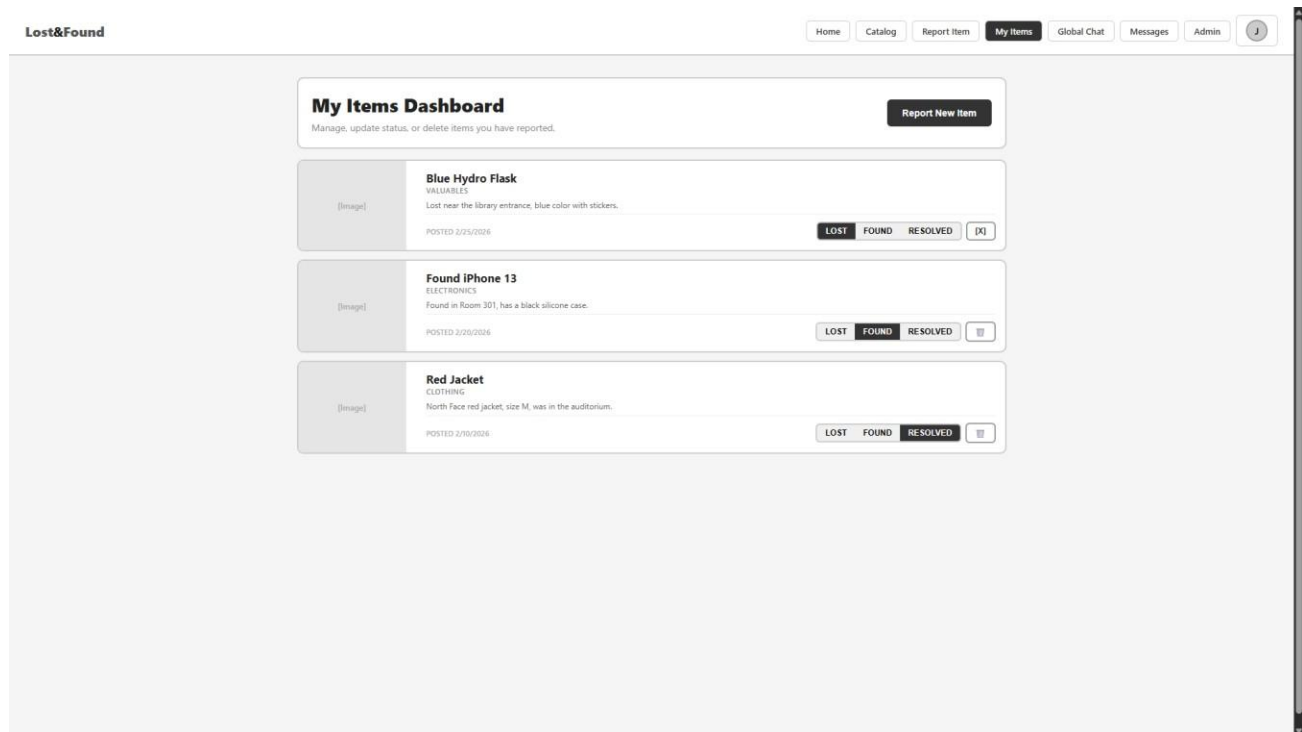
Catalog Page



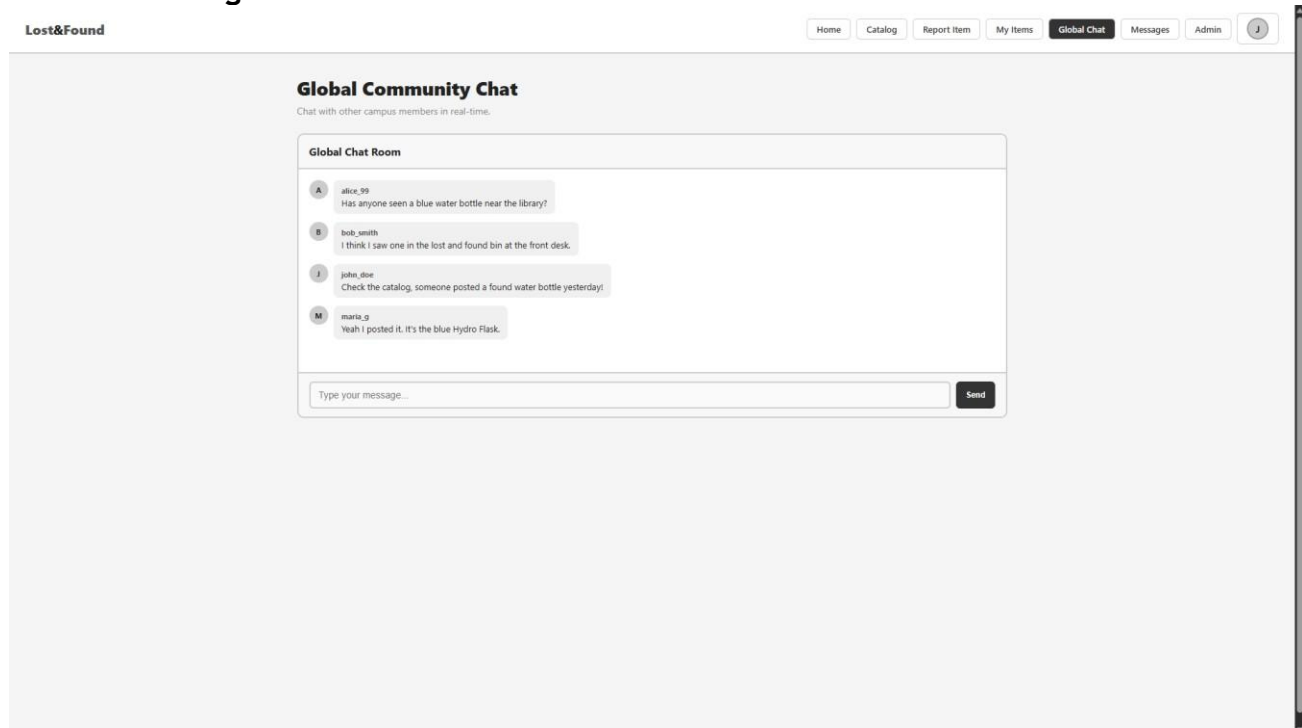
Report Item Page

The screenshot shows the 'Report an Item' form. At the top, there's a navigation bar with links: Home, Catalog, Report Item, My Items, Global Chat, Messages, Admin, and a user profile icon. The main header area includes the title 'Report an Item' and a subtitle 'Fill out the details to report a lost or found item.' The form fields are: Title (with placeholder 'E.g. Blue Hydro Flask or Found iPhone 13'), Status (with buttons 'I lost this' and 'I found this'), Category (with a dropdown menu showing 'Electronics'), Description (with placeholder 'Provide detailed description (color, brand, unique marks...)'), Photo (Optional) (with a dashed box and placeholder '[Upload] Click to upload an image PNG, JPG, JPEG up to 5MB'), Location (with a text input field and placeholder 'Where was this lost/found?'), and an Interactive Map Picker Placeholder (with a large dashed box). At the bottom of the form is a 'Submit Report' button.

Dashboard Page



Global Chat Page



Message Page

Messages

Your private conversations about items.

- M

maria_g Blue Hydro Flask
That's the one! You can file a claim... 2m ago
- B

bob_smith Found iPhone 13
Is the case black silicone? 1h ago
- A

alice_99 Student ID Card
Thanks, I'll pick it up tomorrow 3h ago

M **maria_g**
Re: Blue Hydro Flask

- J

john_doe (You)
Hi, I think the blue Hydro Flask you found is mine!
- M

maria_g
Hey! Can you describe it more? Any stickers?
- J

john_doe (You)
Yes, it has a CIT-U sticker and a cat sticker on the bottom.
- M

maria_g
That's the one! You can file a claim and we'll arrange pickup.
- Type your message...

Send

Admin Dashboard Page

Admin Dashboard

Manage users, warnings, bans, and moderate posts.

[Manage Users](#) [Manage Posts](#)

User Directory

View all registered users and manage their access.

USER	ROLE	STATUS	WARNINGS	ACTIONS
john_doe (email protected)	USER	Active	0 strikes	Warn Ban
alice_99 (email protected)	USER	Active	1 strikes	Warn Ban
bob_smith (email protected)	USER	Banned	3 strikes	Warn Unban
admin_user (email protected)	ADMIN	Active	0 strikes	Warn Ban

Admin Dashboard

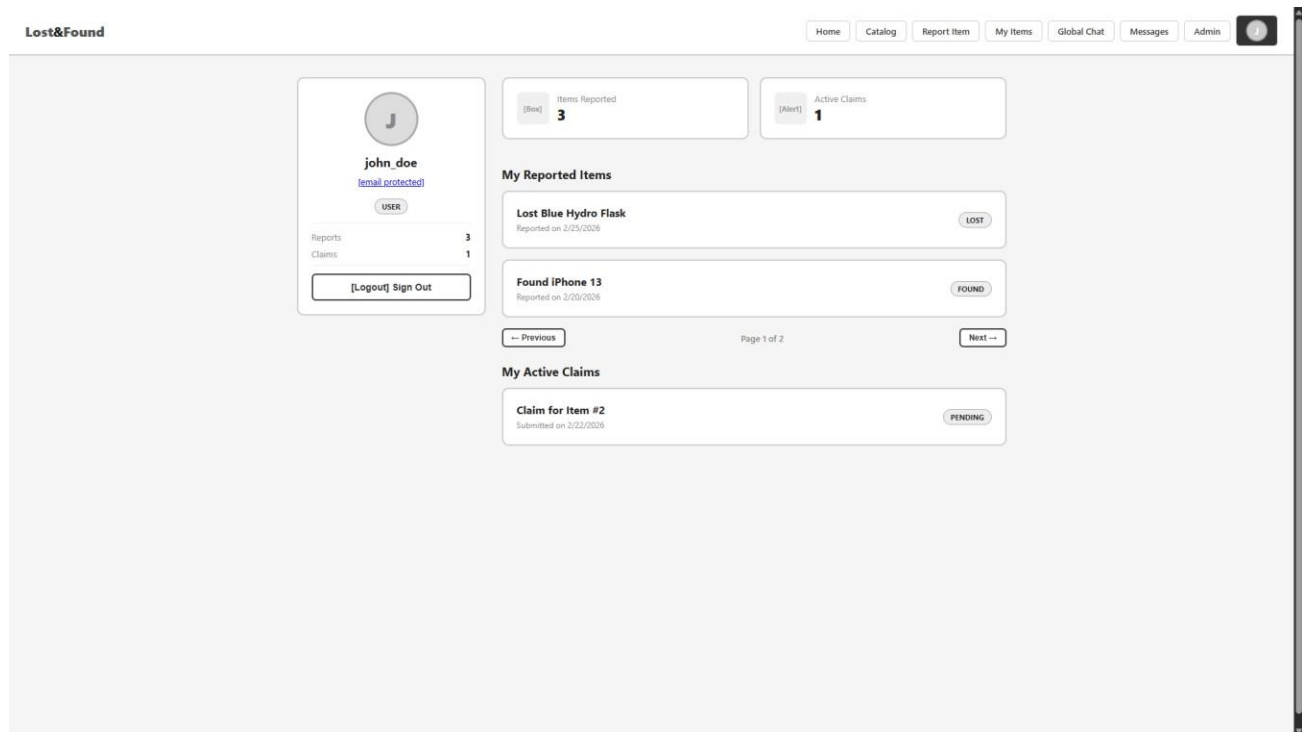
Manage users, warnings, bans, and moderate posts.

[Manage Users](#) [Manage Posts](#)

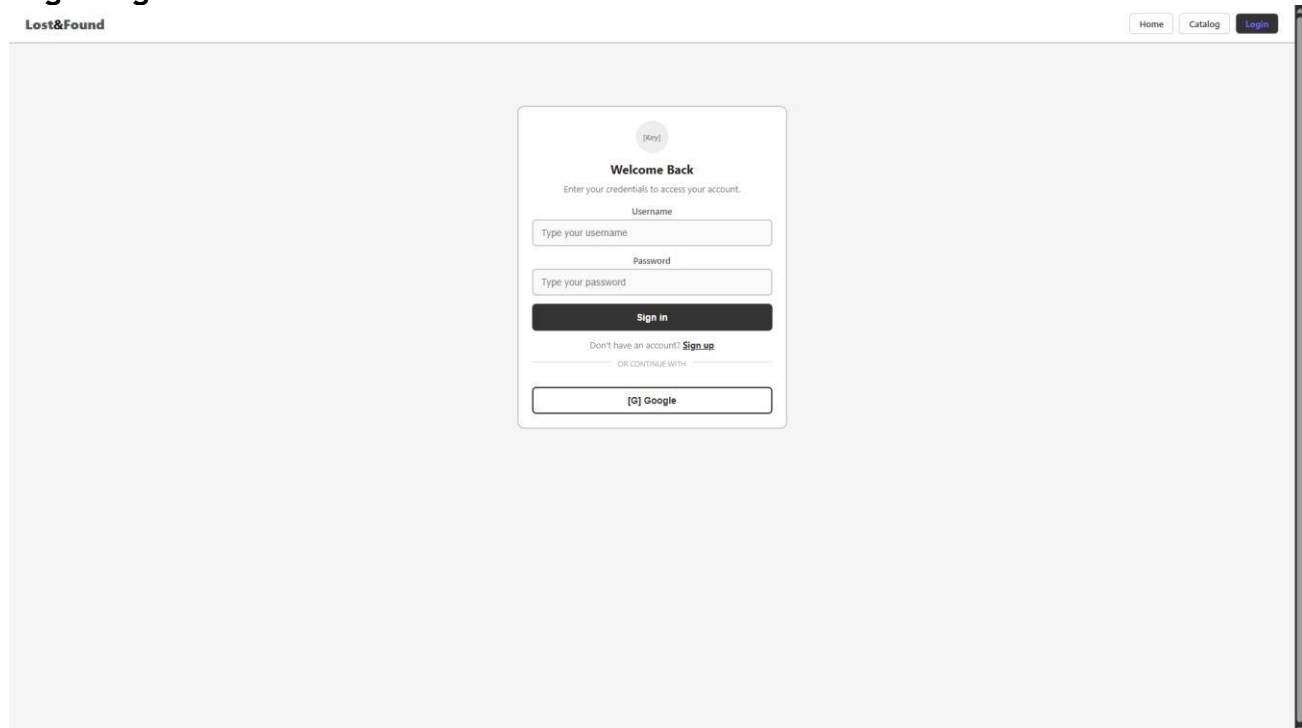
Content Moderation

Delete inappropriate or spam posts permanently.

 Blue Hydro Flask VALUABLES By: john_doe View [X] Delete	 Found iPhone 13 ELECTRONICS By: john_doe View [X] Delete	 Student ID Card DOCUMENTS By: maria_g View [X] Delete
--	---	---



Login Page



Register Page

Lost&Found
Home
Catalog
Login

[User]

Create an Account

Enter your credentials below to get started.

Username

Type your username

Email

Type your email

Password

Type your password

Create Account

Already have an account? [Sign in instead](#)

7.2 Mobile Application Wireframes

Note: This should be wireframes from Figma

Home Page

Lost&Found
☰

New Community Platform

Lost something? Let's find it together.

The smartest, fastest community-driven platform to report found items, track your lost belongings, and securely claim what's yours.

Browse Items →

Report a Find

How it Works

Seamless integration for returning your valuables securely.

[Search]

1. Search & Browse

Search the catalog using rich filters to find your item.

Catalog page

Lost&Found

Lost & Found Hub

Browse and find items that have been reported across campus.

Report an Item

Search

Search by title, description, or location...

AllLostFound

Category:

All Categories

Sort: Newest First

[Image Placeholder]

Blue Hydro Flask

VALUABLES

Lost near the library entrance, blue color with stickers.

LOST2/25/2026

[Image Placeholder]

Report Item Page

Lost&Found

Report an Item

Fill out the details to report a lost or found item.

Title

E.g. Blue Hydro Flask or Found iPhone 13

Status

I lost this

I found this

Category

Electronics

Description

Provide detailed description (color, brand, unique marks...)

Photo (Optional)

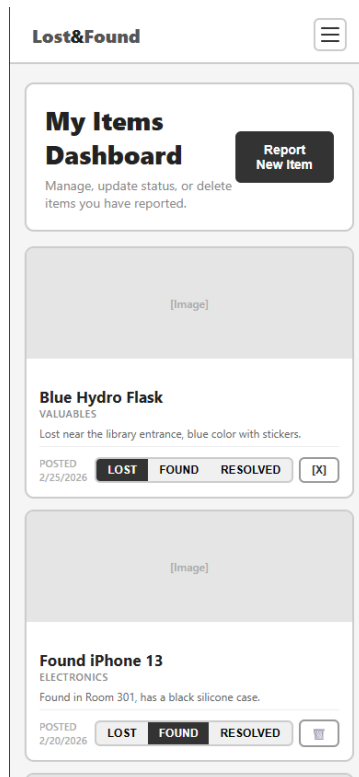
[Upload] Click to upload an image

PNG, JPG, JPEG up to 5MB

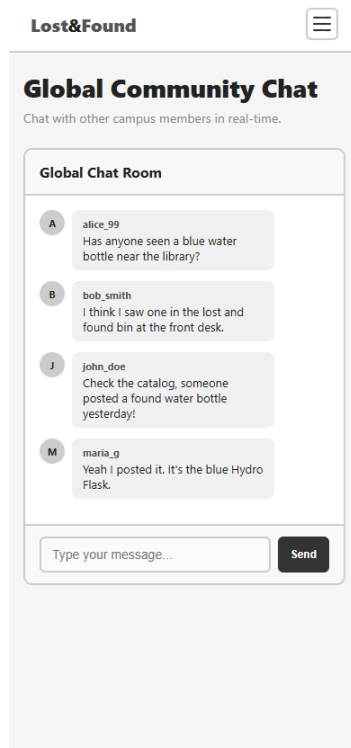
Location

Where was this lost/found?

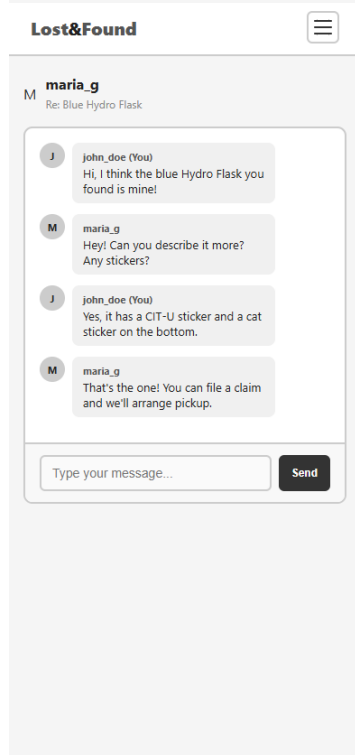
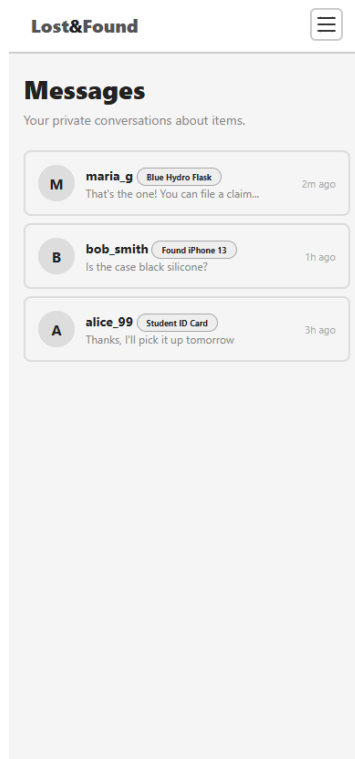
Dashboard Page



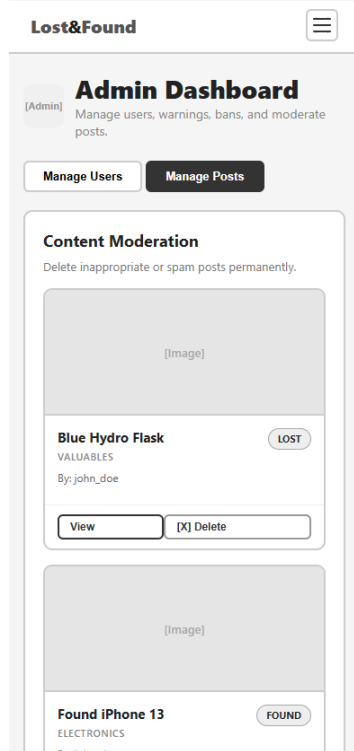
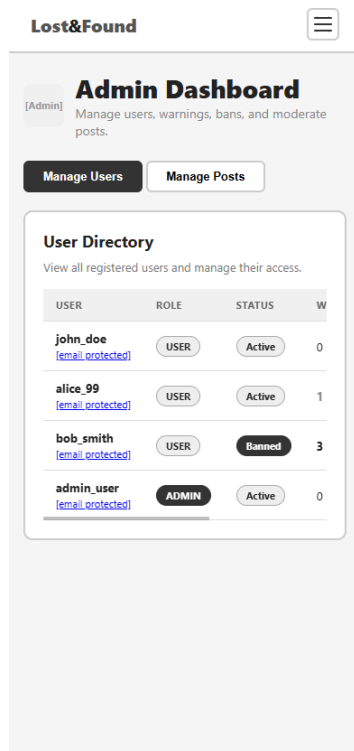
Global Chat Page



Message page



Admin Dashboard Page



Login Page



Lost&Found



Welcome Back

Enter your credentials to access your account.

Username

Type your username

Password

Type your password

Sign in

Don't have an account? [Sign up](#)

OR CONTINUE WITH



Google

Register Page

Lost&Found





Create an Account

Enter your credentials below to get started.

Username

Type your username

Email

Type your email

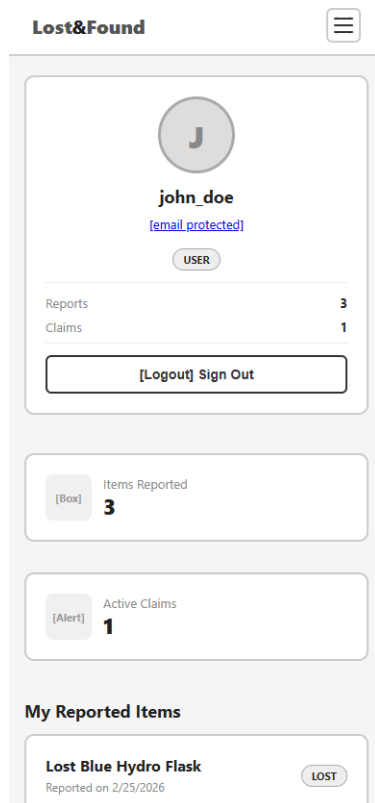
Password

Type your password

Create Account

Already have an account? [Sign in instead](#)

Profile Page



8.0 PLAN

8.1 Project Timeline

Phase 1: Planning & Design (Week 1-2) Week 1: Requirements & Architecture

- Day 1-2: Project setup and documentation
- Day 3-4: Complete FRS and NFR
- Day 5-7: System architecture design

Week 2: Detailed Design

- Day 1-2: API specification (Auth, Items, Claims, Admin, Chat)
- Day 3-4: Database design (Users, Items, Claims, Messages)
- Day 5-6: UI/UX wireframes (Desktop and Mobile)
- Day 7: Implementation plan finalization

Phase 2: Backend Development (Week 3-4) - Week 3: Foundation

- Day 1: Spring Boot setup with dependencies
- Day 2: Database configuration and entity relationships (JPA)

- **Day 3:** JWT authentication and Role-Based Access Control (Admin/User)
- **Day 4:** User management and Profile endpoints
- **Day 5:** Lost & Found Item CRUD operations

Week 4: Core Features

- **Day 1:** Claim functionality and Payment Gateway (Stripe) integration
- **Day 2:** Messaging and WebSocket global chat implementation
- **Day 3:** Search, filtering, Map data, and pagination for Item Catalog
- **Day 4:** Admin dashboard endpoints (Warn, Ban users, Delete posts)
- **Day 5:** API documentation and testing

Phase 3: Web Application (Week 5-6) - Week 5: Frontend Foundation

- **Day 1:** React/Next.js setup with TypeScript and Tailwind CSS
- **Day 2:** Authentication pages (Login, Register)
- **Day 3:** Item Catalog page and React-Leaflet Map integration
- **Day 4:** Item detail and Report Item pages
- **Day 5:** Global Chat and private Messages implementation

Week 6: Complete Web Features

- **Day 1:** File a Claim flow and Stripe UI integration
- **Day 2:** My Items and User Profile dashboard
- **Day 3:** Admin dashboard (Manage Users & Posts)
- **Day 4:** Responsive design polish
- **Day 5:** API integration and testing

Phase 4: Mobile Application (Week 7-8) - Week 7: Android Foundation

- **Day 1:** Android Studio setup, Kotlin, and project structure
- **Day 2:** Authentication screens and JWT handling
- **Day 3:** Item catalog browsing and Native Map view
- **Day 4:** Form to report a lost/found item (with image upload)
- **Day 5:** API service layer implementation using Retrofit

Week 8: Complete Mobile App

- **Day 1:** Claim submission flow and Payment intent handling
- **Day 2:** User profile, global chat display, and reported items management
- **Day 3:** UI polish, XML Layout adjustments, and animations
- **Day 4:** Testing on Android emulator and physical device
- **Day 5:** APK generation and documentation

Phase 5: Integration & Deployment (Week 9-10) Week 9: Integration Testing

- **Day 1:** End-to-end testing across Web and Mobile platforms
- **Day 2:** Bug fixes and optimization
- **Day 3:** Security review (JWT scopes, Admin boundaries)
- **Day 4:** Performance testing
- **Day 5:** Documentation updates

Week 10: Deployment

- **Day 1:** Backend database configuration and deployment (e.g., Railway)
- **Day 2:** Web app deployment (e.g., Vercel)
- **Day 3:** Mobile APK distribution preparation
- **Day 4:** Final live-environment testing
- **Day 5:** Final Project Submission and Defense

Milestones:

- **M1 (End Week 2):** All design documents complete
- **M2 (End Week 4):** Backend Spring Boot API fully functional
- **M3 (End Week 6):** React Web application complete
- **M4 (End Week 8):** Kotlin Mobile application complete
- **M5 (End Week 10):** Full system deployed and integrated

Critical Path:

1. Authentication & API Security system (Week 3)
2. Item Catalog and Map functionality (Week 3-4)
3. Claim functionality and Payment Gateway (Week 4, 6)
4. Mobile integration with existing API using Retrofit (Week 7-8)

5. Cross-platform testing (Week 9)

Risk Mitigation:

- Start with simplest working version of each feature (e.g. text-only posts before image uploads)
 - Test integration points early and often (especially Retrofit API calls from Mobile) •
- Keep backup of working versions
- Focus on core functionality (Authentication, Items, Claims) before enhancements (Global Chat)